CORRIGÉ · PROBLÈME · ANNALES SPÉ

Deux problèmes de communication numérique

Énoncé

1. Analyse d'un texte, dictionnaires et SQL

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Alice compresse un message, l'encode, le transmet ; Bob le décode puis le décompresse. On étudie la compression (codage arithmétique) puis le décodage (algorithme de Viterbi).

Avec le code de longueur fixe `'a'` = 00, `'b'` = 01, `'c'` = 10, la chaîne `s='abaabaca'` tient sur 16 bits. On peut coder `'a'` sur 1 bit et `'b'`, `'c'` sur 2 bits pour obtenir 11 bits.

1. Proposer une telle représentation décodable sans ambiguïté, avec code(`'a'`) < code(`'b'`) < code(`'c'`).

2. Écrire `nbCaracteres(c:str, s:str) -> int` (nombre d'occurrences de `c` dans `s`), linéaire en $n =$ `len(s)`.

3. Que renvoie `listeCaracteres('abaabaca')` ? Expliquer son principe.

```
def listeCaracteres(s:str):
    listeCar = []
    n = len(s)
    for i in range(n):
        c = s[i]
        if not(c in listeCar):
            listeCar.append(c)
    return listeCar
```

4. Complexité dans le pire cas en fonction de n et du nombre k de caractères distincts (le test `c in listeCar` est linéaire en la longueur de `listeCar`).

5. Que fait `analyseTexte` ? Valeur de `analyseTexte('babaaaabca')` ?

```
def analyseTexte(s:str):
    R = []
    l = listeCaracteres(s)
    for i in range(len(l)):
        c = l[i]
        R.append((c, nbCaracteres(c, s)))
    return R
```

6. Complexité de `analyseTexte` en fonction de n et k.

7. Réécrire `analyseTexte` avec un dictionnaire, en complexité linéaire en n, indépendante de k, avec un seul parcours de la chaîne. Exemple : `analyseTexte('abracadabra')` renvoie `{'a':5, 'b':2, 'r':2, 'c':1, 'd':1}`.

Base de données : `caractere(idCar, symbole, typeCaractere, codeHTML)` ; `corpus(idLivre, titre, auteur, annee, nombreCaracteres, langue)` ; `occurrences(idCar, idLivre, nombreOccurrences)`

1. Requête SQL : liste sans doublon des auteurs de `corpus`.

2. Requête SQL : pour chaque caractère, le symbole et sa fréquence en `'Français'` (nombre total d'occurrences divisé par le nombre total de caractères des textes du corpus).

2. Codage arithmétique

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Codage arithmétique : fréquences a : 0,2, b : 0,1, c : 0,2, d : 0,4, e : 0,1, d'où les intervalles a : $[0;0{,}2[$, b : $[0{,}2;0{,}3[$, c : $[0{,}3;0{,}5[$, d : $[0{,}5;0{,}9[$, e : $[0{,}9;1[$. On part de $[0;1[$ et, à chaque caractère, on garde la portion correspondante du sous-intervalle courant. Exemple : `'dac'` donne $[0{,}5;0{,}9[$, puis $[0{,}50;0{,}58[$, puis $[0{,}524;0{,}540[$. `codeCar(car, g, d)` renvoie le sous-intervalle de $[g,d[$ associé à `car`.

1. Intervalle correspondant à `'bac'`.

2. Écrire `codage(s:str) -> (float, float)` qui renvoie l'intervalle final (g,d).

Décodage : on repère dans quelle succession d'intervalles se trouve x. Pour $x = 0{,}123$: `'a'`, puis `'d'` (sous-intervalle $[0{,}10;0{,}18[$), etc.

1. Caractère qui suit `'ad'` pour $x = 0{,}123$, avec le sous-intervalle utilisé.

2. Donner deux chaînes correspondant à 0,2. D'où vient l'ambiguïté ?

On ajoute un caractère de fin `'#'`, de plage non vide près de 0. `decodeCar(x, g, d)` renvoie le caractère dont la plage dans $[g,d[$ contient x.

1. Écrire `decodage(x:float) -> str` (caractère `'#'` compris).

3. Décodage par l'algorithme de Viterbi

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Viterbi. Bob observe `Obs` $= [\text{obs}_0, \dots, \text{obs}_{N-1}]$, symboles de $\Sigma = [\![0, K-1]\!]$. $E[i][j]$: probabilité d'observer i sachant que j a été émis, ici $E = [[0.7, 0.2, 0.3], [0.2, 0.7, 0.1], [0.1, 0.1, 0.6]]$. $P[i][j]$: probabilité d'émettre j à l'instant $t{+}1$ sachant que i a été émis à l'instant t, ici $P = [[0.3, 0.2, 0.5], [0.4, 0.4, 0.2], [0.2$

Graphe : un sommet $S_{i,j}$ par symbole i et par instant j ; arcs de chaque $S_{i,j}$ vers tous les $S_{k,j+1}$; une source σ reliée à la première couche et une cible τ reliée à la dernière. Poids : $\sigma \to S_{i,0}$: $E_{\text{obs}_0,i}$; $S_{i,j} \to S_{k,j+1}$: $E_{\text{obs}_{j+1},k} P_{i,k}$; vers τ : 1. La probabilité d'un chemin est le produit des poids ; on cherche le chemin de probabilité maximale.

1. Nombre de sommets et d'arcs (sans σ, τ et leurs arcs).
2. Pour `Obs = [2,0]`, construire le graphe pondéré.
3. Nombre de chemins de σ à τ ; une exploration exhaustive est-elle envisageable ?
4. Écrire `maximumListe(liste) -> (float, int)` : maximum et plus petit indice où il est atteint.
5. Écrire `glouton(Obs, P, E, K, N) -> [int]`, qui suit à chaque étape l'arc de plus forte probabilité (on dispose de `initialiserGlouton`).
6. Complexité de l'approche gloutonne.
7. Chemin glouton sur la figure 4 (arcs $\sigma \to 0 : 0{,}6$, $\sigma \to 1 : 0{,}4$; $0 \to 0 : 0{,}5$, $0 \to 1 : 0{,}1$, $1 \to 0 : 0{,}9$, $1 \to 1 : 0{,}01$; arcs vers $\tau : 1$). Optimalité ?
8. Ramener la recherche à un plus court chemin à poids positifs ; quel algorithme ?

$T_{i,0} = E_{\text{obs}_0,i}$ et, pour $j \geqslant 1$, $T_{i,j} = \max_k T_{k,j-1} P_{k,i} E_{\text{obs}_j,i}$; `argT` garde l'indice k réalisant le maximum (-1 pour $j = 0$). On dispose de `initialiserViterbi(E, Obs0, K, N)` qui crée `T` et `argT` (K lignes, N colonnes) avec la colonne 0 remplie.

1. Écrire `construireTableauViterbi(Obs, P, E, K, N)` (de bas en haut).
2. Avec les tableaux donnés (dernière colonne de `T` : $1{,}8 \cdot 10^{-5}$, $3{,}5 \cdot 10^{-6}$, $1{,}3 \cdot 10^{-5}$; `argT` lignes : $[-1, 2, 0, 0, 2, 1, 1, 0]$, $[-1, 2, 2, 2, 2, 1, 1, 0]$, $[-1, 2, 0, 0, 2, 1, 1, 0]$), donner la séquence la plus probable.
3. Complexités temporelle et spatiale de la programmation dynamique.

Décoder l'énoncé

Ce que l'énoncé te donne.

- Interdit : `count`, `max`… On écrit les boucles soi-même.
- Le sous-intervalle d'un caractère dans $[g, d[$ est $[g + a(d-g), g + b(d-g)[$ si sa plage dans $[0, 1[$ est $[a, b[$.
- Graphe en N couches de K sommets, arcs complets entre deux couches consécutives.
- Probabilités dans $]0, 1]$: $-\ln$ les rend positives.

Ce qu'on te demande. Compter les caractères d'un texte, d'abord naïvement, puis en un seul parcours avec un dictionnaire ; écrire deux requêtes SQL.

Corrigé

Question 1

On propose :

$\texttt{'a'} \mapsto 0, \qquad \texttt{'b'} \mapsto 10, \qquad \texttt{'c'} \mapsto 11.$

Nombre de bits : `'abaabaca'` contient 5 fois `'a'`, 2 fois `'b'`, 1 fois `'c'` : $5\times1+2\times2+1\times2=11$ bits : `0 10 0 0 10 0 11 0`.

Pas d'ambiguïté : aucun code n'est le début (préfixe) d'un autre (on dit que le code est **préfixe**). On décode en lisant de gauche à droite : un `0` est forcément un `'a'` ; un `1` annonce `'b'` ou `'c'`, et le bit suivant tranche.

Ordre : en binaire, $0 < 10 < 11$ (lus comme $0, 2, 3$), comme demandé.

Question 2

```
def nbCaracteres(c, s):
    compteur = 0
    for i in range(len(s)):
        if s[i] == c:          # une occurrence de plus
            compteur = compteur + 1
    return compteur
```

Une boucle de n tours, chaque tour en temps constant : complexité $O(n)$.

Question 3

On parcourt `'abaabaca'` : `'a'` est ajouté (liste vide), `'b'` est ajouté, les `'a'` et `'b'` suivants sont déjà présents, `'c'` est ajouté.

La fonction renvoie `['a', 'b', 'c']`.

Principe : on parcourt la chaîne et on ajoute chaque caractère à `listeCar` seulement s'il n'y est pas encore. On obtient les caractères distincts, dans l'ordre de leur première apparition.

Question 4

La boucle fait n tours. À chaque tour, le test `c in listeCar` coûte au plus la longueur de `listeCar`, qui est toujours $\leqslant k$ (elle ne contient que des caractères distincts de `s`). Le reste du tour est en temps constant.

Complexité dans le pire cas : $O(nk)$.

Ce pire cas est atteint, par exemple quand les k caractères distincts apparaissent au début puis que la suite répète le dernier : chaque test parcourt alors toute la liste.

Question 5

Rôle : la fonction renvoie la liste des couples (caractère, nombre d'occurrences), un couple par caractère distinct de `s`, dans l'ordre de première apparition.

Exemple `'babaaaabca'` (positions 0 à 9) : `listeCaracteres` donne `['b', 'a', 'c']`. On compte : `'b'` en positions 0, 2, 7 (3 fois) ; `'a'` en positions 1, 3, 4, 5, 6, 9 (6 fois) ; `'c'` en position 8 (1 fois). Vérification : $3 + 6 + 1 = 10$ caractères.

```
[('b', 3), ('a', 6), ('c', 1)]
```

Question 6

- Ligne 3 : `listeCaracteres(s)` coûte $O(nk)$ (Q4).
- Boucle : k tours, chacun appelle `nbCaracteres`, en $O(n)$ (Q2) : en tout $O(nk)$.

Complexité : $O(nk) + O(nk) = O(nk)$.

Question 7

```
def analyseTexte(s):
    compte = {}                 # caractère -> nombre d'occurrences
    for i in range(len(s)):     # un seul parcours de s
        c = s[i]
        if c in compte:         # test en temps constant
            compte[c] = compte[c] + 1
        else:
            compte[c] = 1       # première apparition
    return compte
```

Complexité : n tours, chacun en temps constant (test d'appartenance d'une clé et mise à jour d'un dictionnaire en $O(1)$). Total $O(n)$, indépendant de k, et la chaîne n'est parcourue qu'une fois.

Exemple : `analyseTexte('abracadabra')` renvoie `{'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1}`.

Question 8

```
SELECT DISTINCT auteur
FROM corpus;
```

`DISTINCT` retire les doublons : Zola n'apparaît qu'une fois même s'il a plusieurs livres dans le corpus.

Question 9

Démarche :

- jointure `caractere` – `occurrences` (par `idCar`) pour avoir le symbole, puis `occurrences` – `corpus` (par `idLivre`) pour avoir la langue ;
- on ne garde que les livres en français ;
- on regroupe par caractère et on somme les occurrences ;
- on divise par le nombre total de caractères des livres en français, calculé par une sous-requête.

```
SELECT ca.symbole,
       SUM(o.nombreOccurrences) * 1.0 /
         (SELECT SUM(nombreCaracteres) FROM corpus
          WHERE langue = 'Français') AS frequence
FROM caractere AS ca
JOIN occurrences AS o ON o.idCar = ca.idCar
JOIN corpus AS co ON co.idLivre = o.idLivre
WHERE co.langue = 'Français'
GROUP BY ca.idCar, ca.symbole;
```

Le `* 1.0` force une division décimale (sinon certains systèmes font une division entière, qui donnerait 0). On regroupe par `idCar` (clé primaire) et l'on cite aussi `symbole` pour pouvoir l'afficher. La fréquence est bien entre 0 et 1.

Question 10

Règle : si la plage du caractère dans $[0;1[$ est $[a;b[$, le nouvel intervalle est $[g+a(d-g)\,;\ g+b(d-g)[$.

- `'b'` (plage $[0{,}2;0{,}3[$) dans $[0;1[$: $[0{,}2;0{,}3[$, de longueur $0{,}1$.
- `'a'` (plage $[0;0{,}2[$) dans $[0{,}2;0{,}3[$: $[0{,}2+0\times 0{,}1\,;\ 0{,}2+0{,}2\times 0{,}1[= [0{,}2;0{,}22[$, de longueur $0{,}02$.
- `'c'` (plage $[0{,}3;0{,}5[$) dans $[0{,}2;0{,}22[$: $[0{,}2+0{,}3\times 0{,}02\,;\ 0{,}2+0{,}5\times 0{,}02[$.

$$\texttt{'bac'} \longmapsto [0{,}206\,;\ 0{,}21[.$$

Vérification : la longueur finale vaut $0{,}1\times 0{,}2\times 0{,}2 = 0{,}004$, produit des fréquences.

Question 11

```
def codage(s):
    g, d = 0, 1                    # intervalle de départ [0, 1[
    for i in range(len(s)):
        g, d = codeCar(s[i], g, d)  # on affine avec le caractère lu
    return (g, d)
```

Exemple : `codage('dac')` renvoie $(0{,}524\,;\,0{,}540)$.

Question 12

Après `'ad'`, l'intervalle courant est $[0{,}10; 0{,}18[$, de longueur $0{,}08$. La position relative de x vaut

$$\frac{x - 0{,}10}{0{,}08} = \frac{0{,}023}{0{,}08} \approx 0{,}29 \in [0{,}2; 0{,}3[,$$

plage de `'b'`. Le sous-intervalle correspondant est

$$[0{,}10 + 0{,}2 \times 0{,}08\,;\, 0{,}10 + 0{,}3 \times 0{,}08[= [0{,}116\,;\, 0{,}124[\ni 0{,}123$$: le caractère suivant est `'b'`.

Question 13

$0{,}2 \in [0{,}2; 0{,}3[$: 0,2 code `'b'`. Mais 0,2 est aussi la borne gauche de l'intervalle de `'ba'`, $[0{,}2; 0{,}22[$, et de celui de `'baa'`, $[0{,}2; 0{,}202[$, etc.

`'b'` et `'ba'` (et aussi `'baa'`, `'baaa'`…) correspondent toutes à 0,2.

Origine : le réel ne dit pas où la chaîne s'arrête. Comme la plage de `'a'` commence à 0, ajouter des `'a'` à la fin garde la même borne gauche : on ne sait pas quand arrêter le décodage.

Question 14

```
def decodage(x):
    s = ''
    g, d = 0, 1
    c = ''
    while c != '#':                # '#' marque la fin de la chaîne
        c = decodeCar(x, g, d)     # caractère dont la plage contient x
        s = s + c
        g, d = codeCar(c, g, d)    # on descend dans son sous-intervalle
    return s
```

La boucle s'arrête au premier `'#'` lu, qui est inclus dans la chaîne renvoyée, comme demandé. (Avec des flottants de précision limitée, cela ne marche que pour des chaînes courtes, comme le rappelle l'énoncé.)

Question 15

Sommets : N couches (une par observation), K sommets par couche : NK sommets.

Arcs : entre deux couches consécutives, chacun des K sommets est relié aux K sommets suivants : K^2 arcs. Il y a $N - 1$ paires de couches consécutives.

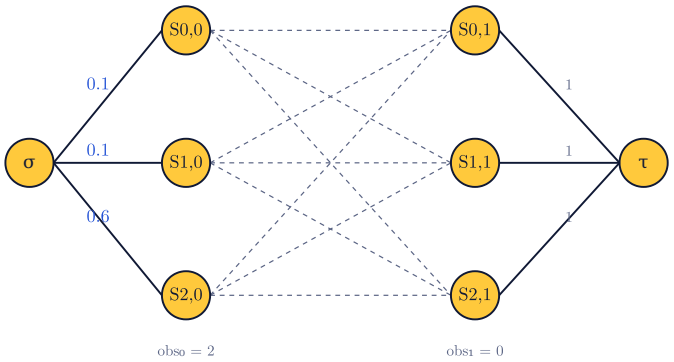
$$NK \text{ sommets}, \qquad (N-1)K^2 \text{ arcs}.$$

Question 16

Arcs issus de σ : $E_{\text{obs}_0,i} = E_{2,i}$, ligne 2 de E : 0,1 ; 0,1 ; 0,6.

Arcs internes $S_{i,0} \to S_{k,1}$: $E_{\text{obs}_1,k} P_{i,k} = E_{0,k} P_{i,k}$, avec la ligne 0 de E : (0,7 ; 0,2 ; 0,3).

de \{} vers	$S_{0,1}$	$S_{1,1}$	$S_{2,1}$
$S_{0,0}$	$0{,}7 \times 0{,}3 = 0{,}21$	$0{,}2 \times 0{,}2 = 0{,}04$	$0{,}3 \times 0{,}5 = 0{,}15$
$S_{1,0}$	$0{,}7 \times 0{,}4 = 0{,}28$	$0{,}2 \times 0{,}4 = 0{,}08$	$0{,}3 \times 0{,}2 = 0{,}06$
$S_{2,0}$	$0{,}7 \times 0{,}2 = 0{,}14$	$0{,}2 \times 0{,}3 = 0{,}06$	$0{,}3 \times 0{,}5 = 0{,}15$



Arcs depuis σ : $E_{2,i}$. Les neuf arcs du milieu sont pondérés par $E_{0,k} P_{i,k}$ (tableau ci-dessus).

Arcs vers τ : poids 1.

Question 17

Un chemin de σ à τ choisit exactement un sommet dans chacune des N couches, et tous les choix sont possibles (arcs complets) : il y a K choix par couche, donc

$$K^N \text{ chemins.}$$

C'est exponentiel en N : pour $K = 3$ et un message de seulement $N = 100$ symboles, $3^{100} \approx 5 \times 10^{47}$ chemins. Une exploration exhaustive est **impossible** en pratique.

Question 18

```
def maximumListe(liste):
    vmax, imax = liste[0], 0
    for i in range(1, len(liste)):
        if liste[i] > vmax:      # strict : on garde le premier indice du maximum
            vmax, imax = liste[i], i
    return (vmax, imax)
```

L'inégalité stricte garantit que l'on renvoie le **plus petit** indice réalisant le maximum. On suppose la liste non vide.

Question 19

Remarque sur le code fourni : dans `initialiserGlouton`, il faut lire `E[Obs[0]][i]` (ligne de l'observation, colonne du symbole) ; `E[Obs[0][i]]` est une coquille.

```
def glouton(Obs, P, E, K, N):
    etats = [initialiserGlouton(Obs, E, K)]    # meilleur premier symbole
    for j in range(1, N):
        i = etats[-1]                           # symbole courant
        probas = [E[Obs[j]][k] * P[i][k] for k in range(K)]  # poids des K arcs sortants
        p, k = maximumListe(probas)             # arc le plus probable
        etats.append(k)
    return etats
```

Question 20

L'initialisation examine K valeurs. Puis $N-1$ tours de boucle ; à chaque tour, on calcule K produits et on cherche le maximum de K valeurs : $O(K)$.

Complexité : $O(NK)$.

Question 21

Chemin glouton : depuis σ, l'arc le plus lourd mène au symbole 0 ($0{,}6 > 0{,}4$) ; depuis ce sommet, l'arc le plus lourd reste sur 0 ($0{,}5 > 0{,}1$). Chemin $\sigma \to 0 \to 0 \to \tau$, de probabilité $0{,}6 \times 0{,}5 \times 1 = 0{,}30$.

Meilleur chemin : $\sigma \to 1 \to 0 \to \tau$, de probabilité $0{,}4 \times 0{,}9 \times 1 = 0{,}36 > 0{,}30$. (Les deux autres valent $0{,}6 \times 0{,}1 = 0{,}06$ et $0{,}4 \times 0{,}01 = 0{,}004$.)

Le glouton renvoie $[0, 0]$ alors que l'optimum est $[1, 0]$: l'approche gloutonne n'est **pas optimale**. Un premier choix un peu moins bon pouvait ouvrir un arc bien meilleur.

Question 22

On remplace chaque poids $p \in]0, 1]$ par $w = -\ln p \geqslant 0$. Pour un chemin de poids $p_1, \dots, p_r$:

$$\sum_{\ell} w_\ell = -\ln\Big(\prod_{\ell} p_\ell\Big).$$

Comme $-\ln$ est strictement décroissante, maximiser le produit des probabilités revient à **minimiser la somme des poids** w, qui sont positifs (un arc de probabilité nulle est simplement retiré). On cherche donc un plus court chemin de σ à τ dans un graphe à poids positifs : on peut utiliser l'**algorithme de Dijkstra**.

(Le graphe étant sans circuit et rangé en couches, une simple programmation dynamique couche par couche suffit : c'est l'algorithme de Viterbi qui suit.)

Question 23

Ordre de calcul : la relation donne la colonne j à partir de la colonne $j-1$ seulement. On remplit donc les colonnes de gauche à droite (de bas en haut au sens de la programmation dynamique), la colonne 0 étant déjà remplie par `initialiserViterbi`.

```
def construireTableauViterbi(Obs, P, E, K, N):
    T, argT = initialiserViterbi(E, Obs[0], K, N)
    for j in range(1, N):            # colonne j, à partir de la colonne j-1
        for i in range(K):           # symbole i à l'instant j
            valeurs = [T[k][j-1] * P[k][i] * E[Obs[j]][i] for k in range(K)]
            T[i][j], argT[i][j] = maximumListe(valeurs)   # meilleur prédécesseur k
    return T, argT
```

Question 24

Dernière colonne ($j = 7$) de `T` : $1{,}8 \cdot 10^{-5}$, $3{,}5 \cdot 10^{-6}$, $1{,}3 \cdot 10^{-5}$. Le maximum est en ligne 0 : le dernier symbole est 0.

Remontée : `argT[i][j]` donne le symbole à l'instant $j-1$.

- $j = 7$, symbole 0 : `argT[0][7]` $= 0$, donc symbole 0 en $j = 6$;
- $j = 6$, symbole 0 : `argT[0][6]` $= 1$, donc 1 en $j = 5$;
- $j = 5$, symbole 1 : `argT[1][5]` $= 1$, donc 1 en $j = 4$;
- $j = 4$, symbole 1 : `argT[1][4]` $= 2$, donc 2 en $j = 3$;
- $j = 3$, symbole 2 : `argT[2][3]` $= 0$, donc 0 en $j = 2$;
- $j = 2$, symbole 0 : `argT[0][2]` $= 0$, donc 0 en $j = 1$;
- $j = 1$, symbole 0 : `argT[0][1]` $= 2$, donc 2 en $j = 0$ (et `argT[2][0]` $= -1$: on est arrivé à la source).

Séquence la plus probable : $[2, 0, 0, 2, 1, 1, 0, 0]$.

Question 25

Temps : il y a $(N-1)K$ cases à calculer ; chacune demande K produits et un maximum sur K valeurs, soit $O(K)$. Avec l'initialisation en $O(NK)$:

$$\text{temps} : O(NK^2), \qquad \text{espace} : O(NK) \text{ (les deux tableaux } K \times N).$$

C'est polynomial, à comparer aux K^N chemins de Q17.

Le piège

1. En Q9, diviser par `COUNT(*)` ou oublier de restreindre le total aux livres en français : on n'obtiendrait plus une fréquence.
2. En Q10, multiplier les bornes au lieu de multiplier la longueur : le nouvel intervalle est décalé de g et mis à l'échelle de $d-g$.
3. En Q24, lire `argT` dans le mauvais sens : `argT[i][j]` est le symbole à l'instant $j-1$, pas à l'instant $j+1$.

La méthode

1. Pour compter en un seul parcours : un dictionnaire de compteurs. Pour une complexité : nombre de tours de boucle multiplié par le coût d'un tour, sans oublier les tests d'appartenance cachés.
2. Intervalle emboîté : $[g, d[\to [g + a(d-g), g + b(d-g)[$. Pour décoder, on refait le même chemin en cherchant à chaque étape la plage qui contient x.
3. Chemin optimal dans un graphe en couches : programmation dynamique couche par couche (valeur optimale + prédécesseur), puis remontée depuis la meilleure case de la dernière couche.

Rappels de cours utilisés

- **Complexité d'une boucle.** On compte les opérations élémentaires : une boucle de n tours dont le corps coûte $O(c)$ coûte $O(nc)$. Deux boucles imbriquées de n et k tours : $O(nk)$. Un test `x in L` sur une liste coûte $O(\text{len}(L))$; sur un dictionnaire, $O(1)$ en moyenne.
- **Dictionnaires en Python.** `d = {}` crée un dictionnaire vide ; `c in d` teste la présence d'une clé ; `d[c] = v` crée ou modifie ; `d[c] += 1` incrémente une valeur existante.
- **SQL : agrégation et jointure.** `SELECT DISTINCT` retire les doublons. `JOIN ... ON` relie deux tables par une clé. `GROUP BY` regroupe les lignes, et `SUM`, `COUNT` calculent sur chaque groupe. Une sous-requête entre parenthèses peut fournir une valeur.
- **Algorithme glouton.** À chaque étape, il fait le choix qui paraît le meilleur sur le moment, sans jamais revenir en arrière. Il est rapide, mais pas toujours optimal : un contre-exemple suffit à le prouver.
- **Plus court chemin.** Dans un graphe à poids positifs, l'algorithme de Dijkstra calcule les plus courts chemins depuis une source. Le logarithme transforme un produit en somme :

$\ln(ab) = \ln a + \ln b$, et $-\ln p \geqslant 0$ quand $0 < p \leqslant 1$.

- **Programmation dynamique.** On écrit une relation de récurrence entre sous-problèmes, on remplit un tableau dans l'ordre où les valeurs nécessaires sont déjà calculées (de bas en haut), et on garde de quoi reconstruire la solution (les « prédécesseurs »).